

Submission Date: February 18, 2026

Project Milestone #2
4+1 View Model + Database
Connection

SFWRTECH 4SA3 – Software Architecture

Submitted By:

Student Name: Amisha

Student ID: 400612546

Submitted To:

Professor Name: Kevin Browne

Learning Objectives

- Design and document a software architecture using the 4+1 view model.
- Implement a software architecture, including design patterns.
- Present a software architecture design.

Spendly – Smart Expense Tracker

Project Purpose:

Spendly is a smart yet simple expense tracking software that will be designed to help users record, organize, and review their personal spending. The purpose of the software is to allow users to log daily expenses, categorize them, keep track of spending over time, and view summarized reports such as monthly or category-based spending totals, including simple expected income and expense summaries based on recurring transactions. Spendly will focus on providing a minimal and easy-to-use system that supports basic accounting needs without unnecessary complexity, allowing non-business users to better understand their spending habits and manage their finances more effectively.

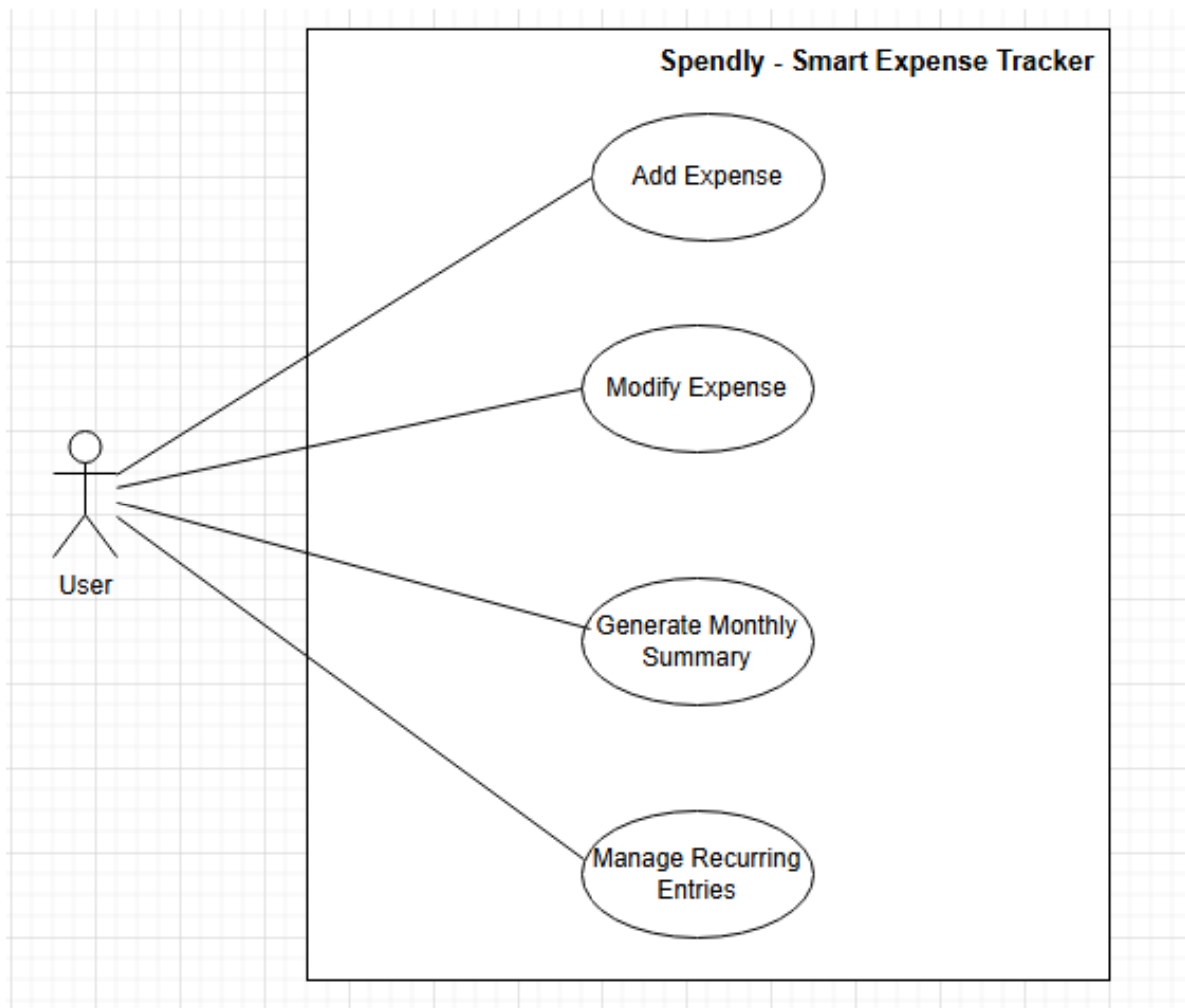
Target Audience:

The target audience for Spendly is non-accounting users such as students, working professionals, and individuals who manage personal finances on a regular income. The software is intended for people who want a simple way to track their expenses, understand where their money is going, and stay aware of upcoming income and recurring costs without needing accounting knowledge or complex financial tools.

Requirements:

ID	Name	Type	Description
R1	Add Expense	Functional	The system allows a user to add a new expense record including amount, date, and category.
R2	Modify Expense	Functional	The system allows a user to update or delete an existing expense record.
R3	Generate Monthly Summary	Functional	The system generates a monthly summary of total recorded expenses.
R4	Manage Recurring Entries	Functional	The system allows users to define recurring income and recurring expense entries.
R5	Cloud Data Storage	Non-Functional	The system stores and retrieves data using a cloud-based database service.
R6	Architectural Separation	Non-Functional	The system architecture maintains separation between user interface, business logic, and data storage components.

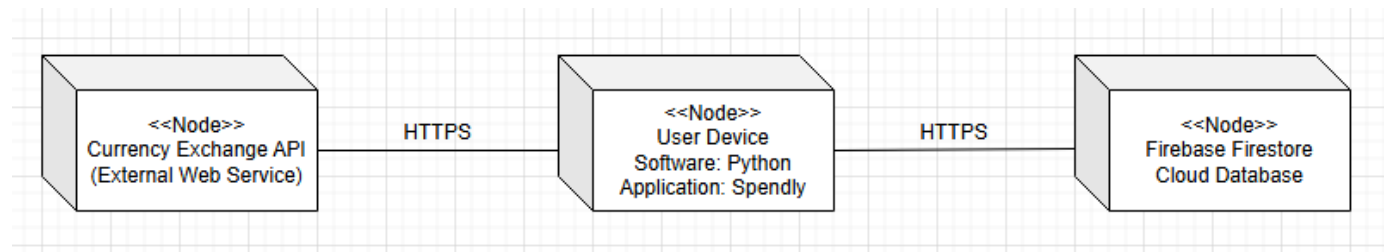
Scenario Viewpoint - UML Case Diagram:



Explanation: This Scenario Viewpoint above shows the way a user interacts with the Spendly system at a high level. In this diagram, the primary actor is the User using the application and they interact directly with the system to perform key financial tracking tasks. They can add expenses, modify existing expense records, generate monthly summaries, and manage recurring income and expense entries.

This viewpoint focuses on the system's functional requirements from the user's perspective. It highlights the main services that the system provides without showing internal implementation details or database interactions. The purpose of this view is to demonstrate how the system supports user goals through clearly defined use cases.

Physical Viewpoint - Deployment Diagram:



Explanation: This Physical Viewpoint above represents the deployment structure of Spendly system. The application runs on a user device as a Python-based program. This device hosts the Spendly application, which contains the core business logic and user interaction components.

It then communicates with Firebase Firestore, which serves as the cloud-based database responsible for storing and retrieving expense, income, and recurring transaction data. This communication occurs over HTTPS via the internet.

In addition, the application connects to a third-party currency exchange rate API using HTTPS. This external service provides up-to-date exchange rate information when currency conversion is required.

This viewpoint focuses on how system components are distributed across physical environments and demonstrates the interaction between the local application, cloud database, and external web services.

Cloud Database Connection Demonstration Code:

To demonstrate access to the selected cloud database, a simple Python script was created to connect to Firebase Firestore. The script initializes the Firebase application using the provided credentials and performs a basic data operation, such as adding or retrieving a sample expense record.

The purpose of this code is to verify that the Spendly application can successfully establish a connection to the cloud database and perform basic data manipulation.

❖ *Source code is added as .zip file in Avenue to Learn*